

Efficient Conjunctive Geometric Range Query Over Encrypted Spatial Data With Learned Index

Mingyue Li , Chunfu Jia , Ruizhong Du , and Guanxiong Ha 

Abstract—With the increasing popularity of geo-positioning technologies and mobile Internet, spatial data query services have attracted extensive attention. To protect the confidentiality of sensitive information outsourced to cloud servers, much efforts have been devoted to designing geometric range query schemes over encrypted spatial data without affecting availability. However, existing works focus on the privacy-preserving schemes with traditional tree indexes, causing more computing and storage issues. In this paper, we propose an efficient conjunctive geometric range query scheme over encrypted spatial data with a learned index. In particular, we design a new privacy-preserving learned index for spatial data to reduce the search space and storage overhead. The main idea is to add noise disturbance to the objective function instead of directly adding it to output results, reducing the leakage of private information and ensuring the correctness of output results. Moreover, we propose a spatial segmentation algorithm to avoid accessing a large number of unnecessary Z codes in the query process. The formal security analysis shows that our scheme ensures index data security and query privacy. Simulation results show that the query efficiency is improved while the storage overhead is significantly reduced compared with the state-of-the-art schemes.

Index Terms—Privacy-preserving, conjunctive geometric range query, encrypted spatial data, learned index.

I. INTRODUCTION

THE pervasiveness of a location-based service results in the explosive growth of spatial data, since smartphones and wearable devices can produce a large amount of geospatial data every day through the use of daily applications such as Uber, Google Maps, and Facebook [1]. To effectively process the massive spatial data, many enterprises and organizations

provide users with geometric range query (GRQ) services that can return the spatial data falling into geometric objects. For instance, users can find friends in the circular area around their location. GRQ is now widely used because it can assist people's daily activities (e.g., taking an Uber) and play an important role in some public health and safety applications.

Driven by the cloud-first initiative, outsourcing spatial data to cloud service providers (such as Amazon RedShift) has become a common practice. The powerful computing power and huge storage space of cloud servers can significantly reduce data management costs for enterprises and organizations and improve the search efficiency of original spatial data [2]. However, the sensitive information of users (such as location information) may be leaked [3], [4], which brings enormous losses to users' digital assets and incurs substantial social impact. According to the statistics of the United Nations Trade and Development Organization (UNCTAD), as of February 21, 2022, about 80% of the world's countries have completed data security and privacy legislation or have proposed draft laws¹. Therefore, to prevent data privacy leakage, data owners encrypt the data before outsourcing them to cloud servers. Nevertheless, this limits the availability of spatial data because the retrieval technologies on plaintext cannot be used to query encrypted data [5]. Subsequently, a line of research has been made to design geometric range query schemes [6], [7], [8], [9], [10], [11], [12], [13], [14] over encrypted spatial data to protect data confidentiality without affecting data availability.

A. Problem and Motivation

As shown in Table I, in most existing schemes [6], [7], [8], [9], [10], [12], [13], [14], the search intention of users cannot be accurately expressed in practical applications, since they only support geometric range queries and a user often wants to query data points that meet some attributes within a geometric range. Unfortunately, to the best of our knowledge, only a few privacy-preserving boolean range query schemes are proposed in [11], [14]. More importantly, the above schemes use traditional tree indexes (i.e., Quadtree [11], [14], R-tree [8], [10], [13], Binary tree [12]) to query large-scale spatial datasets, which causes computing and storage issues:

- *Low query efficiency.* For the above encrypted indexes with traditional tree structures, each query needs to access the encrypted vectors stored by all nodes on the path

¹<https://unctad.org/page/data-protection-and-privacy-legislation-worldwide>

Received 30 July 2024; revised 31 October 2024; accepted 24 August 2025. Date of publication 2 September 2025; date of current version 10 November 2025. This work was supported in part by the National Key R&D Program of China under Grant 2018YFA0704703; in part by the National Natural Science Foundation of China under Grant 61972073 and Grant 62172238; in part by the Natural Science Foundation of Hebei Province of China under Grant F2023201033; and in part by the Natural Science Foundation of Hebei Education Department of China under Grant QN2024204. Recommended for acceptance by K. Chakraborty. (Corresponding author: Chunfu Jia.)

Mingyue Li and Ruizhong Du are with the School of Cyber Security and Computer, Hebei University, Baoding 071002, China, and also with the Key Laboratory of High Trusted Information System in Hebei Province, Baoding 071002, China.

Chunfu Jia and Guanxiong Ha are with the College of Cyber Sciences, Nankai University, Tianjin 300350, China (e-mail: cfjia@nankai.edu.cn).

Digital Object Identifier 10.1109/TC.2025.3604470

TABLE I
COMPARISON WITH EXISTING SCHEMES

Schemes	CP	AP	MD	CGRQ	IS
GRSE-tree [8]	BNG	✗	✓	✗	R-Tree
FastGeo [9]	SSW	✗	✗	✗	Quadtree
EGRQ [12]	OPE	✗	✓	✗	R-Tree
I [12]	ASHE	✓	✗	✗	Binary tree
II [13]	ASHE	✓	✗	✗	Binary tree
[13]	HE	✓	✗	✗	✗
PBRQ-T [14]	KSW	✗	✓	✓	✗
PBRQ-T ⁺ [14]	KSW	✗	✓	✓	Quadtree
ECGRQ-LI	SHVE	✓	✓	✓	Learned index

AP represents the short for “Access Pattern”; MD represents the short for “Multiple Dimensions”; CGRQ represents the short for “Conjunctive Geometric Range Query”; CP represents the short for “Cryptographic Primitive”; IS represents the short for “Index Structure”; ASHE represents the short for “Additive Symmetric Homomorphic Encryption”; ✓ denotes that the presentation scheme has this function, ✗ denotes that the scheme does not have this capability.

from the tree root to a leaf node and perform ciphertext calculations. Therefore, a large number of indirect queries limit the search performance.

- *High storage overhead.* The use of a traditional tree index requires additional space higher than the original data storage space to store the index. For example, the research shows that the index occupies 55% of the total memory space in the commercial memory database [15], which is intolerable for large-scale datasets.

Therefore, we explore the potential of the learned index in geometric range queries over encrypted spatial data. The concept of the learned index was first proposed in [16], which can generate a special index structure with high adaptability by learning the data distribution of search keys. When a search key is input, the learned index predicts the positions of the records where the key is located. It can improve search efficiency and simplify the index structure. However, existing learned indexes [16], [17], [18], [19], [20] do not consider the issue of privacy protection, which cannot be directly applied to design privacy-preserving search schemes, since the cloud servers can spy on spatial data according to the input and output of a learned model. In addition to attacks on the learned models, there are also reconstruction attacks using the pattern leakages of range queries to recover the underlying search keywords. For instance, Falzon et al. [21] proposed a reconstruction attack using access pattern leakages, which can accurately reconstruct the entire index set after observing $O(N_0 + N_1)(RN^2 + R \log R)$ queries for a database D with R records over a rectangular $N_0 \times N_1$ domain, where $N = N_0 N_1$.

B. Our Contributions

We propose a privacy-preserving learned index structure for spatial data objects to design a conjunctive geometric range query scheme, improving retrieval efficiency, reducing storage overheads, and protecting data privacy. Our main contributions can be summarized as follows.

- To improve the efficiency of geometric range queries over encrypted spatial data and reduce the index storage cost, we propose a privacy-preserving learned index structure through expanding the objective function by polynomials and adding noise to each coefficient, which can protect data privacy by obscuring the access patterns of range queries and avoid the reconstruction attack in [21].
- We further design an Efficient Conjunctive Geometric Range Query scheme with the proposed Learned Index, namely ECGRQ-LI. In particular, to decrease the search space and matching time, the concept of spatial segmentation is introduced to avoid accessing a large number of irrelevant Z codes.
- We conduct extensive simulation experiments on real data. The results show that the ECGRQ-LI scheme is much more efficient than the PBRQ-T⁺ scheme of [14], which is the state-of-the-art scheme supporting conjunctive range queries. Specifically, the query efficiency of the ECGRQ-LI scheme is $32 - 47\times$ faster than the PBRQ-T⁺ scheme. The data storage overhead of ECGRQ-LI is significantly reduced, saving 66% memory compared with PBRQ-T⁺. In addition, we also remove the attribute vector in ECGRQ-LI and compare it with the most effective geometric range query scheme in [12]. Concretely, the efficiency of ECGRQ-LI is $2.1 - 3.3\times$ faster than [12] and the memory is saved by 59%.

II. RELATED WORK

A. Geometric Range Query

To realize secure circular range queries, Zhu et al. [6] used the BGN scheme [22] to calculate the distance and hash table to calculate the result range, but the homomorphic operation in BGN led to great computational overhead. Therefore, Wang et al. proposed [7], [8], [9] schemes based on SSW encryption [23] to avoid expensive homomorphic encryption. However, due to the use of a great quantity pairing operations in SSW, the performances of the above schemes are limited. Xu et al. [10] used a secure k-nearest neighbor algorithm and polynomial fitting technology to implement geometric range queries and utilized R-tree to reduce matching time. Wang et al. [11] proposed a boolean range query scheme (PBRQ) based on symmetric-key hiding vector encryption (SHVE) [24] and gray codes for encrypted spatial data. Besides, they designed a bitmap quadtree (BQ-tree) index structure to reduce the search complexity of PBRQ. Subsequently, Tong et al. [14] further proposed a PBRQ scheme supporting temporal access control through attribute encryptions and forward and backward derivations. However, the scheme still uses quadtree to build indexes. Very recently, Kermanshahi et al. proposed two geometric range query schemes I and II in [12] by constructing binary tree index and proposed Geo-DRS scheme in [13] by constructing R⁺ tree. However, using the traditional tree index makes the index data larger than the basic data set. This problem is more serious when the data have more spatial dimensions. In addition, a large number of internal tree nodes need to be accessed in the query process, which affects the search efficiency.

B. Learned Index

Kraska et al. [16] first proposed the concept of the learned index. The core idea is to build a model that can learn the data distribution of search keys and effectively predict the location of records, improving query efficiency and reducing data storage costs greatly. For example, a B-tree index takes $O(\log n)$ search time and $O(n)$ index memory size, while a learning model might achieve $O(1)$ search time and $O(1)$ index memory size. Subsequently, dynamic learning index models are proposed in [17], [18], [19], [25] as the model in [16] is static. When the data distribution of leaf nodes changes, all connected upper nodes of leaf nodes need to be retrained. However, the above research on the learned index is limited to one-dimensional ordered data. To support efficient queries of spatial data, Wang et al. [20] designed a new spatial index structure, ZM learned index, by using Z-order spatial filling curve and a hierarchical learned model in [16]. However, a large number of irrelevant Z codes are accessed in the query process, resulting in poor query performance of the model. More importantly, the above learned indexes do not consider the issue of privacy protection.

C. Searchable Encryption

Searchable encryption (SE) is a technology that supports users to search for keywords in ciphertext, which was first proposed by Song et al. [26], and has since sparked extensive excellent research. To provide secure update operations, efficient result verifications, and prevent key leakage, Yang et al. [27] proposed a verifiable multi-client dynamic searchable encryption scheme, which mainly solves the above problems by designing a policy hidden index, state-based update method, and delegated result verification mechanism. To address the performance issues when searchable symmetric encryption (SSE) is combined with SGX, Jiang et al. [1] proposed a novel SSE structure. This structure groups keywords and stores these groups using different Bloom filters that were previously loaded into the Enclave in order, achieving a secure and effective conjunctive keyword search. Zhao et al. [28] proposed a two-stage data retrieval scheme, which combines distributed trust, autonomous random access, and smart contracts to prevent the leakage of access patterns. To ensure the confidentiality of vehicle data using SE technology, Yang et al. [29] proposed a privacy-preserving data sharing framework based on blockchain technology for vehicles. Specifically, they designed a chain-based access control scheme and a non-interactive search token generation scheme to accelerate access control and search processes, respectively. The existing searchable encryption schemes can provide good ideas for designing privacy-preserving spatial data query schemes, but they cannot be directly applied.

III. PRELIMINARIES

A. Notations

In Table II, we define the notations used in our manuscript.

TABLE II
NOTATIONS AND DESCRIPTIONS

$Z_{\{P\}}$:	The encrypted value of the z-code of point P .
t :	The length of Z codes.
s :	The dimensions of attribute vectors.
D :	The spatial data set, denotes $D = \{D_1, D_2, \dots, D_m\}$.
a_i :	The attribute values extracted from the spatial data D_i .
Ip :	The $(t + s)$ -dimensional vector corresponding to D_i .
Ip^* :	The encrypted form of vector Ip .
A_D :	The attribute set of the spatial data.
n :	The size of the attribute set A_D .
A_P :	The attribute vector of point P .
Q :	The $(t + s)$ -dimensional query vector, denotes $Q = \{Q_1, Q_2, \dots, Q_{t+s}\}$.
Q^* :	The encrypted form of vector Q .
TQ :	The trapdoor for a query $\{G, q\}$, denotes $TQ = \{Q^*, [Z_{\{p^l\}}], [Z_{\{p^r\}}]\}$.
p^l :	The point of the lower-left corner of query region G .
p^r :	The point of the upper-right corner of query region G .

B. Z-Order Spatial Filling Curve

To perform the conjunctive geometric range queries on spatial data, Z-order spatial filling curves in [20] are introduced to reduce the dimension of spatial data. The formal definition of the Z-order curve is as follows.

Definition 1: The Z-order curve is the mapping of the n -dimensional space R_n and one-dimensional space R , which is defined as $R_n \rightarrow R$. If a point $P \in R_n$, then $Z_{\{P\}} \in R$, that is, $Z_{\{P\}}$ is the space filling curve code of the point P . For the set $\{P_1, P_2, \dots, P_N\}$, there is $Z\{P_1, P_2, \dots, P_N\} = \{Z_{\{P_1\}}, \dots, Z_{\{P_N\}}\}$, where $Z\{P\}$ represents the process of encoding point P .

Given a $T \times T$ grid, each spatial data point can be converted to a unique Z code, the length of Z codes is $t = 2(\lceil \log_2 T \rceil)$, where Z codes are ordered and calculated by interleaving operations on grid coordinates. Therefore, spatial data objects can be sorted according to the corresponding Z codes. Particularly, artificial neural network models can be used to learn the data distribution of Z codes and construct the learned index.

C. Differential Privacy

Differential privacy [30] is a technology designed to provide a way to maximize the accuracy of the data query and minimize the opportunity to identify its records when querying from a statistical database. Its formal definition is as follows:

Definition 2: Differential privacy. For an algorithm $M : D \rightarrow R$, when all possible outputs $S \in R$ for M and adjacent data sets D and D' can meet $Pr[M(D) \in S] \leq e^\epsilon Pr[M(D') \in S]$, then the algorithm M is ϵ -differential privacy, where the smaller the value of ϵ is, the stronger the intensity of privacy protection is. In addition, for any query function f , the sensitivity $\Delta f = \max_{D, D'} \|f(D) - f(D')\|$ represents the maximum absolute distance in the output of the query function for adjacent data sets, which can provide a basis for adding disturbances. Moreover, suppose there are n algorithms $M_1, M_2, \dots, M_n$, and their privacy budgets are $\epsilon_1, \epsilon_2, \dots, \epsilon_n$, respectively. For the data set D , the combined algorithm $M = \{M_1(D), M_2(D), \dots, M_n(D)\}$ satisfies $(\sum_1^n \epsilon_i)$ -differential privacy.

D. Predicate Encryption

We use the symmetric-key predicate encryption in [24] as the cryptographic tool of the ECGRQ-LI scheme since it can support effective conjunctive queries without disclosing privacy.

Definition 3: The predicate encryption scheme in [24] can evaluate whether two vectors match without leaking privacy, which consists of four basic algorithms.

- 1) $PE.Setup(1^\lambda) \rightarrow MSK$: Input security parameter λ , output master secret key MSK and message space χ .
- 2) $PE.Encrypt(MSK, I_P \in \Gamma^{t+s}, \mu \in \chi) \rightarrow I_P^*$: Input key MSK , message μ in message space χ and vector $I_P = (I_{P_1}, \dots, I_{P_{t+s}}) \in \Gamma^{t+s}$, output the ciphertext I_P^* associated with (I_P, μ) , where $\Gamma^{t+s} \in \{0, 1\}$ is a finite set and does not contain wildcard symbols.
- 3) $PE.GenToken(MSK, Q \in \Gamma_{*}^{t+s}) \rightarrow SK$: Input key MSK and query vector $Q = (Q_1, \dots, Q_{t+s}) \in \Gamma_{*}^{t+s}$, output decryption key SK , where Γ_{*}^{t+s} contains wildcard symbols.
- 4) $PE.Query(I_P^*, SK) \rightarrow \mu / \perp$: Input ciphertext I_P^* corresponding to the index vector I_P and the decryption key SK corresponding to the query vector Q , output μ if $P_{I_P^*}^{PE}(Q) = 1$; otherwise $\perp$. For each $I_P \in \Gamma^{t+s}$, $Q \in \Gamma_{*}^{t+s}$, we have

$$P_{I_P^*}^{PE}(Q) = \begin{cases} 1, & I_{P_i} = Q_i \text{ or } Q_i = *, \forall 1 \leq i \leq t+s \\ \perp, & \text{otherwise} \end{cases}$$

E. Pseudorandom Function

For a function $F: K \times X \rightarrow Y$, it is a pseudorandom function if for any adversary A , the advantage can be defined as $Adv_A^F(\lambda) = |Pr[A^{F(K, \cdot)}(1^\lambda) = 1] - Pr[A^{f(\cdot)}(1^\lambda) = 1]| \leq \text{negl}(\lambda)$, where $\text{negl}(\lambda)$ is negligible in λ and $f(\cdot)$ is a real random function.

IV. SYSTEM MODELS, THREAT MODELS, AND SECURITY DEFINITIONS

A. System Model

As shown in Fig. 1, the ECGRQ-LI scheme mainly consists of three entities, namely spatial data owner (SDo), cloud service provider (CSP), and spatial data user (SDu).

SDo is mainly responsible for submitting encrypted spatial data and a generated secure learned index to CSP to improve retrieval efficiency. In addition, SDo formulates corresponding access control policies to authorize query users. CSP has powerful computing and storage capabilities to store encrypted spatial data and indexes. When receiving the conjunctive geometry range query requests from a legitimate SDu, the CSP performs the calculation on the encrypted data and returns the corresponding search results to the SDu. The SDu generates a trapdoor (i.e., encrypted search request) and sends it to CSP. After receiving the search results, the SDu decrypts these query results using the key sent by SDo.

B. Threat Model

Honest-but-curious servers. CSP is an honest-but-curious entity that provides reliable services and tries to infer private

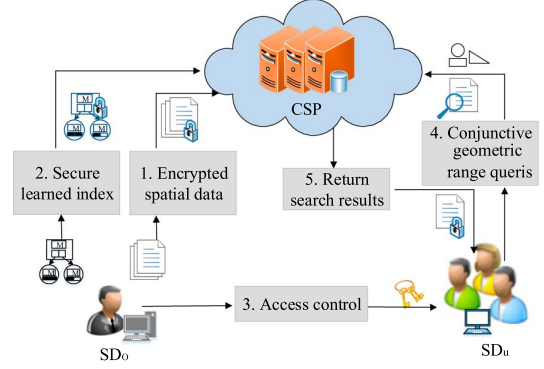


Fig. 1. System model.

information about spatial data objects and conjunctive geometric range queries.

Honest authorized users. Similar to most spatial query schemes [31], SDu is considered to be trusted in this paper who neither actively discloses the plaintext of the data owner to an unauthorized entity nor actively discloses the obtained decryption information.

C. Security Definitions

We first define four leakage functions named $\mathcal{L}_{size}$, $\mathcal{L}_{precision}$, $\mathcal{L}_{access}$, and $\mathcal{L}_{search}$ before presenting the formal security definition of ECGRQ-LI. $\mathcal{L}_{size} = \{n_I, n_Q\}$ is the leakage of size patterns, where n_I denotes the total number of encrypted indexes, n_Q denotes the total number of conjunctive geometry range queries (i.e., trapdoors) submitted. $\mathcal{L}_{precision} = \{g, g'\}$ is the leakage of precision patterns, where g and g' are the code length of a spatial data point and trapdoor. $\mathcal{L}_{access} = \text{Access}(Q)$ is the leakage of access patterns, where $\text{Access}(Q) = \{D'_1, D'_2, \dots, D'_i\} (i \in (1, m))$ is the encrypted data returned for an encrypted conjunctive geometry range query Q . $\mathcal{L}_{search} = \text{Search}(Q)$ is the leakage of search patterns, where $\text{Search}(Q) = \{e_1, e_2, \dots, e_{g'}\}$ denotes the search pattern and an adversary can learn whether an encryption data point is returned by different query trapdoors. The value of e_i is 1 or 0, which indicates whether two queries retrieve the same key. For index dataset I and query Q , let $\mathcal{L}(I, Q) = \{\mathcal{L}_{size}, \mathcal{L}_{precision}, \mathcal{L}_{access}, \mathcal{L}_{search}\}$.

The main security requirements of ECGRQ-LI are to protect the privacy of queries and index data stored on the CSP. We then present the security definitions for index privacy and query privacy of ECGRQ-LI with the above leakage functions, which strictly follow Indistinguishable under Selective Chosen-Plaintext Attacks (IND-SCPA) [32], [33].

Suppose $\Pi = (\text{Setup}, \text{IndexBuild}, \text{TrapGen}, \text{Query})$ is a probabilistic ECGRQ-LI scheme based on the security parameter λ , the security game between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$ is defined as Fig. 2.

Definition 4: The ECGRQ-LI scheme ensures the index privacy under IND-SCPA, if the adversary $\mathcal{A}$ in the $\text{Index}_{\mathcal{A}}(\lambda)$ game has the negligible advantages at most:

$$Adv_{\text{Index}}^{\text{ECGRQ-LI}}(1^\lambda) = |Pr[b' = b] - 1/2| \leq \text{negl}(\lambda)$$

Index Privacy Game $Index_A(\lambda)$.

Initialization: The challenger $\mathcal{C}$ runs $Setup(1^\lambda)$ to generate the secret key Key and keeps it private. The adversary $\mathcal{A}$ sends two plaintext index datasets $D_0 = \{D_{0,1}, \dots, D_{0,i}, \dots, D_{0,m}\}$ and $D_1 = \{D_{1,1}, \dots, D_{1,i}, \dots, D_{1,m}\}$ with the same number of index records to the challenger $\mathcal{C}$.

Requests: $\mathcal{A}$ adaptively selects multiple query requests and sends them to $\mathcal{C}$. The requests include the following two types:

- *Ciphertext:* For the j -th ciphertext request, the adversary $\mathcal{A}$ sends $D_j = \{D_{j,1}, \dots, D_{j,i}, \dots, D_{j,m}\}$ to the challenger $\mathcal{C}$. The challenger responds with $\{IP_j^*, [I_j]\} \leftarrow IndexBuild(Key, D_j)$.

- *Trapdoor:* For the j -th trapdoor request, $\mathcal{A}$ sends the conjunctive geometric range query $Q_j = \{G_j, q_j\}$. $\mathcal{C}$ responds with $TQ_j \leftarrow TrapGen(Key, Q_j)$, where $\mathcal{L}(D_0, Q_j) = \mathcal{L}(D_1, Q_j)$.

Challenge: For D_0 and D_1 , $\mathcal{C}$ randomly selects $b \in \{0, 1\}$, and calculates with $\{IP_b^*, [I_b]\} \leftarrow IndexBuild(Key, D_b)$ to $\mathcal{A}$. $\mathcal{A}$ continues to submit a series of requests adaptively.

Guess: The adversary $\mathcal{A}$ guesses that the value of b is $b' \in \{0, 1\}$.

Query Privacy Game $Query_A(\lambda)$.

Initialization: The challenger $\mathcal{C}$ runs $Setup(1^\lambda)$ to generate the secret key Key and keeps it private. The adversary $\mathcal{A}$ sends two plaintext conjunctive geometric range queries Q_0 and Q_1 with the same dimension to $\mathcal{C}$.

Requests: $\mathcal{A}$ adaptively selects multiple query requests and sends them to the challenger $\mathcal{C}$. The requests include the following two types:

- *Ciphertext:* For the j -th ciphertext request, $\mathcal{A}$ sends $D_j = \{D_{j,1}, \dots, D_{j,i}, \dots, D_{j,m}\}$ to the challenger $\mathcal{C}$. Subsequently, the challenger responds with $\{IP_j^*, [I_j]\} \leftarrow IndexBuild(Key, D_j)$, where $\mathcal{L}(D_{j,i}, Q_0) = \mathcal{L}(D_{j,i}, Q_1)$.

- *Trapdoor:* For the j -th trapdoor request, $\mathcal{A}$ sends the conjunctive geometric range query $Q_j = \{G_j, q_j\}$. The challenger $\mathcal{C}$ responds with $TQ_j \leftarrow TrapGen(Key, Q_j)$.

Challenge: For Q_0 and Q_1 , $\mathcal{C}$ randomly selects $b \in \{0, 1\}$, and calculates with $Q_b^* \leftarrow TrapGen(SK, Q_b)$ to $\mathcal{A}$. $\mathcal{A}$ continues to submit a series of requests adaptively.

Guess: The adversary $\mathcal{A}$ guesses that the value of b is $b' \in \{0, 1\}$.

Fig. 2. The security game between $\mathcal{A}$ and $\mathcal{C}$.

Definition 5: The ECGRQ-LI scheme ensures the query privacy under IND-SCPA, if the adversary $\mathcal{A}$ in the $Query_A(\lambda)$ game has the negligible advantages at most:

$$Adv_{Query}^{ECGRQ-LI}(1^\lambda) = |Pr[b' = b] - 1/2| \leq negl(\lambda)$$

V. PRIVACY-PRESERVING LEARNED INDEX

A. Problem Definition

Learned indexes can utilize the distribution of spatial data to construct learned models that can predict the locations of corresponding data, achieving efficient search and minimal storage overhead. However, the existing schemes have not taken into account the issue of privacy protection. Therefore, we propose a new Privacy-preserving learned index Model for spatial data objects, called PM learned index, whose core idea is to encode spatial data and learn the data distribution of encrypted Z-codes to construct learned models. In addition, to further protect the privacy of the learned models, the differential privacy technique is used to add noise to the loss function for the output disturbance. Compared with directly adding noise to the output results, our method can use the optimization and adaptation of the learned model to correct the deviation of the optimal solution caused by noise addition. It can interfere with the effective information contained in the output results as much

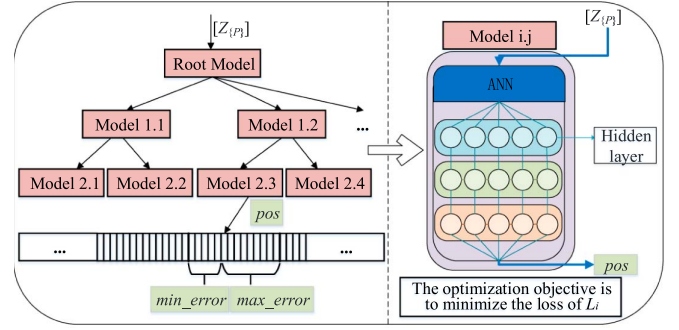


Fig. 3. Hierarchical learning index model.

as possible to reduce the leakage of private information while ensuring the correctness of the output results.

B. Deployment of Differential Privacy Mechanism

As shown in Fig. 3, the PM index is a hierarchical learned model consisting of multiple layers of artificial neural networks. Each layer is responsible for predicting the subset of records that satisfy a specific range query. Moreover, the PM index takes an encrypted Z code as the input, selects the model of the next layer, and repeats this process until the last layer predicts the positions. Specifically, assuming that x is an encrypted Z code to be queried, y is the corresponding position of x ($y \in (0, m)$), the model of the first stage is recorded as f_1 , then the loss function L_1 is expressed as:

$$L_1 = \sum_{(x,y)} (f_1(x) - y)^2 \quad (1)$$

Assuming that the number of models in layer i ($i > 1$) is $|M|_i$, the j -th model in layer i is expressed as $f_i^{(j)}$, and the loss function of model $f_i^{(j)}$ is

$$L_i = \sum_{(x,y)} (f_{i-1}^{(\lfloor |M|_i f_{i-1}(x)/N \rfloor)}(x) - y)^2 \quad (2)$$

where $f_{i-1}(x) = f_{i-1}^{(\lfloor |M|_{i-1} f_{i-2}(x)/N \rfloor)}$. For $1 \leq i \leq l$, each layer can be recursively trained by using the loss function L_i to build a complete model, l is the number of layers of the learned index model.

To protect the privacy of spatial data, the straightforward way is to add noise to the input and output layers of the PM model to achieve differential privacy. However, the method of disturbing the input data generally uses differential privacy technology to synthesize the data, which consumes a lot of computing resources. In addition, directly disturbing the output results may bring a large error rate. Therefore, a function mechanism is considered to solve the above problems. The main idea is to add noise disturbance to the objective function of the optimization problem instead of directly adding it to the output result. The optimization and adaptation of the learned model can be used to correct the deviation of the optimal solution caused by the noise addition. In addition, the function mechanism is an extension of the Laplace mechanism and can be directly deployed on various objective functions. Therefore,

we use the Laplace mechanism to disturb the function L_i since the loss function is the objective function of the optimization problem and plays an important role in the privacy of the whole model. The specific method is to expand the objective function by polynomials and add noise to each coefficient. Assuming that the obtained polynomial is $\Phi_i(x)$ and the sensitivity is Δf , then $\Delta f = 2\max \sum_{j=1}^N \sum_{p \in \Phi_{i,j}} \|\lambda_p\|$, where λ_p represents the coefficient of the function $p(x)$ in the polynomial $\Phi_i(x)$, $N \in [0, \infty]$. Besides, given a privacy parameter ε_i , the noise distribution is:

$$p(\eta) = \frac{\varepsilon_i}{2\Delta f} \exp\left(-|\eta| \cdot \frac{\varepsilon_i}{\Delta f}\right) \quad (3)$$

Definition 6 (Differential privacy of PM learned index): For any encrypted Z code set with a length of m , $[Z_{\{P\}}] \in \Delta^m$, any adjacent training data sets D and $D' \in 2^{2^\Delta}$, and any leakage $\mathcal{L}$, the PM model satisfies ε -DP if the following formula holds, where $\varepsilon = \sum_{i=1}^l \varepsilon_i$.

$$Pr[PM(D, [Z_{\{P\}}]) = \mathcal{L}] \leq e^{m\varepsilon} Pr[PM(D', [Z_{\{P\}}]) = \mathcal{L}]$$

Let M represent the random mechanism, which inputs the training dataset D and the Z code $[Z_{\{P\}}]$ of the spatial data point P , and outputs the location range corresponding to $[Z_{\{P\}}]$ in the server. Let D, D' be two adjacent datasets, which are same except for the k -th spatial data point. Let $[Z_{\{P\}}]^*$ and $[Z_{\{P\}}]^{*'} represent encrypted Z codes that exist only in $D[k]$ and $D'[k]$ respectively. $pos[k]$ and $pos'[k]$ are the outputs of $M(D, [Z_{\{P\}}]^*)$ and $M(D', [Z_{\{P\}}]^{*'})$, respectively. Let p be the result observed by an adversary. Then we need to find the upper bound of the ratio rat , where $rat = \frac{Pr(M(D, [Z_{\{P\}}]^*)=p)}{Pr(M(D', [Z_{\{P\}}]^{*'})=p)} =$$

$$\frac{Pr(pos=p)}{Pr(pos'=p)} = \frac{\prod_{i=1}^m Pr(pos[i]=p[i])}{\prod_{i=1}^m Pr(pos'[i]=p[i])}.$$

If $[Z_{\{P\}}] \notin \{[Z_{\{P\}}]^*, [Z_{\{P\}}]^{*'}\}$, the ratio above is 1.

If $[Z_{\{P\}}] = [Z_{\{P\}}]^*$, the following derivation can be obtained according to the relevant proof in [34]. $\frac{Pr(M(D, [Z_{\{P\}}]^*)=p)}{Pr(M(D', [Z_{\{P\}}]^{*'})=p)} =$

$$\prod_{D[k] \neq D'[k]} \frac{pos[k]=p[k]}{pos'[k]=p[k]} \leq \frac{a+b(1-a)}{b}, \text{ where } a \text{ denotes the probability of } M(DB, [Z_{\{P\}}]) \in [p - min_{error}, p + max_{error}] \text{ and } b \text{ denotes the probability of } M(DB, [Z_{\{P\}}]) \notin [p - min_{error}, p + max_{error}], a > b.$$

When $[Z_{\{P\}}] = [Z_{\{P\}}]^{*'}$, we can prove the boundary according to the same process.

$$\frac{Pr(M(D, [Z_{\{P\}}]^{*'})=p)}{Pr(M(D', [Z_{\{P\}}]^*)=p)} = \prod_{D[k] \neq D'[k]} \frac{pos[k]=p[k]}{pos'[k]=p[k]} \leq \frac{1}{1-a}.$$

In summary, the ratio $rat = \frac{Pr(M(D, [Z_{\{P\}}])=p)}{Pr(M(D', [Z_{\{P\}}])=p)} \leq \prod_{D[i] \neq D'[i]} \max\{1, \frac{a+b(1-a)}{b}, \frac{1}{1-a}\} = \frac{a+b(1-a)}{b} \Rightarrow \varepsilon = \ln \frac{a+b(1-a)}{b}.$

Therefore, the PM model can meet ε -differential privacy, where the privacy parameter $\varepsilon = \ln \frac{a+b(1-a)}{b}$. It is worth noting that ε is not a fixed value because a and b are variables, and users can adjust ε by setting different values of a and b . The smaller the value of ε , the more noise added. Therefore, with the increase of ε , the degree of privacy protection becomes stronger. On the contrary, the accuracy of the point query is reduced.

²The encrypted Z-codes of position i in dataset D and D' are completely different, that is, $[Z_{\{P\}}]$ is in $D[i]$ or $D'[i]$ but not both.

Algorithm 1 *Point_Search*

Require: Learned index PM , a search point P ;

Ensure: Predicted position pos .

```

1:  $Z_{\{P\}} \leftarrow Z\{P\}$   $\triangleleft$  Compute the Z code of the point  $P$ 
2:  $[Z_{\{P\}}] \leftarrow F(K, Z_{\{P\}})$ 
3: for  $1 \leq i < l$  do
4:    $Model[i+1] \leftarrow Model[i]_{Prediction}([Z_{\{P\}}])$ 
5:    $i = i + 1$ 
6: end for
7:  $pos = Model[l]_{Prediction}([Z_{\{P\}}])$   $\triangleleft$  Prediction results
8:  $range = [pos - min_{error}, pos + max_{error}]$ 
9: return  $pos, range$ 

```

C. Secure and Efficient Point Query

As shown in Algorithm 1, we can use the PM learned index to perform secure and efficient point queries of spatial data, which is the basis for range query of spatial data. For a point P , the corresponding encrypted Z code $[Z_{\{P\}}]$ is calculated and input into the PM index for search. The position estimation is obtained at each layer for selecting the model at the next layer. Compared with the traditional index trees, there is no search process between each layer of the PM index, which can avoid the low search efficiency caused by accessing a large number of internal tree nodes. Besides, there are certain errors in the process of location estimation, that is, the location estimated in the last layer may not be the exact location of the query. However, it is very close to the real location of the data object, which can ensure that the given search $[Z_{\{P\}}]$ must be within the range of $[pos - min_{error}, pos + max_{error}]$. Due to the use of differential privacy technology, the output location ranges are different for two identical inputs, obfuscating the access patterns for point queries.

VI. OUR CONSTRUCTION

A. Problem Definition

In this section, our goal is to design an efficient conjunctive geometric range query scheme by the PM learned index proposed in Section V. The main challenge is to avoid accessing unnecessary Z-codes during the query process and reduce search space and matching time. For example, for the G_1 query in Fig. 5, the SDu can perform the query operation through accessing the Z codes between the point p^{rh} in the upper-right corner and the point p^{ll} in the lower-left corner. However, using the above method directly for query G_2 will result in accessing a large number of irrelevant Z codes and affect the query efficiency. Therefore, we propose a spatial segmentation method that divides the query G_2 into several sub-queries according to the way of dividing the space $\mathcal{P}_D$, and then search them respectively. The specific segmentation algorithm is described in Algorithm 2, where the most critical step is to segment $\mathcal{P}_D$.

B. Construction Overview

Suppose n attributes are extracted from the spatial data set and attribute i contains s_i attribute values, then each spatial

Algorithm 2 $Segment(\mathcal{P}_D, G_S) \rightarrow \{[Z_{\{p^{ll}\}}], [Z_{\{p^{rh}\}}]\}$

Require: Whole plane area $\mathcal{P}_D$, geometric range G_S ;

Ensure: Encrypted Z codes $[Z_{\{p^{ll}\}}]$ and $[Z_{\{p^{rh}\}}]$.

```

1: Divide the  $x$ -axis into  $T_x$  partitions evenly
2: Calculate the variance  $v$  of each partition for the  $y$ -axis
3: if  $v > Threshold$  then
4:   for  $1 \leq i \leq T_x$  do
5:     Divide each partition  $i$  along the  $y$ -axis to generate  $T_y$ 
       partitions
6:   end for
7: else
8:   Divide the  $y$ -axis into  $T_y$  partitions evenly
9: end if
10:  $\mathcal{P}_D \leftarrow \{\mathcal{P}_{D_1}, \mathcal{P}_{D_2}, \dots, \mathcal{P}_{D_\theta}\} | \theta = T_x \times T_y$ 
11: for  $1 \leq i \leq \theta$  do
12:    $\mathcal{P}_{D_i} \leftarrow \{p_{D_i}^{rh}, p_{D_i}^{ll}\}$ 
13: end for
14:  $G_S \leftarrow \{p_S^{rh}, p_S^{ll}\}$ 
15:  $[Z_{\{p^{rh}\}}] \leftarrow \phi, [Z_{\{p^{ll}\}}] \leftarrow \phi$ 
16: if  $(p_S^{rh} \in \mathcal{P}_{D_i}) \wedge (p_S^{ll} \in \mathcal{P}_{D_i}) |_{i \in (1, \theta)}$  then
17:    $\triangleright$  The points in the upper right corner and the lower left
       corner of  $G_S$  are in the same partition
18:    $[Z_{\{p^{rh}\}}] \leftarrow [Z_{\{p^{rh}\}}] \cup [Z_{\{p_S^{rh}\}}]$ 
19:    $[Z_{\{p^{ll}\}}] \leftarrow [Z_{\{p^{ll}\}}] \cup [Z_{\{p_S^{ll}\}}]$ 
20: else
21:    $i = 1$ 
22:   while  $(p_S^{rh} \notin \mathcal{P}_{D_j}) \wedge (p_S^{ll} \in \mathcal{P}_{D_j}) |_{j \in (1, \theta)}$  do
23:     if  $(p_i^{rh} \in \mathcal{P}_{D_j}) \wedge (Z_{\{p^{rh}\}} < Z_{\{p_i^{rh}\}}++)$  then
24:        $\triangleright$  The point  $p_i^{rh}$  is the point at the upper right corner
           of the region  $G_{S_i}$ 
25:        $p_{S_i}^{ll} \leftarrow p_{S_i}^{ll}, p_{S_i}^{rh} \leftarrow p_i^{rh}$ 
26:        $[Z_{\{p^{rh}\}}] \leftarrow [Z_{\{p^{rh}\}}] \cup [Z_{\{p_{S_i}^{rh}\}}]$ 
27:        $[Z_{\{p^{ll}\}}] \leftarrow [Z_{\{p^{ll}\}}] \cup [Z_{\{p_{S_i}^{ll}\}}]$ 
28:        $[Z_{\{p^{rh}\}}] \leftarrow [Z_{\{p^{rh}\}}] \cup [Z_{\{p_{S_i}^{rh}\}}]$ 
29:     end if
30:      $S = S - S_i$ 
31:      $i = i + 1$ 
32:   end while
33:    $\gamma \leftarrow i$ 
34:    $[Z_{\{p^{rh}\}}] \leftarrow [Z_{\{p^{rh}\}}] \cup [Z_{\{p_{S_i}^{rh}\}}]$ 
35:    $[Z_{\{p^{ll}\}}] \leftarrow [Z_{\{p^{ll}\}}] \cup [Z_{\{p_{S_i}^{ll}\}}]$ 
36: end if
37: return  $[Z_{\{p^{rh}\}}], [Z_{\{p^{ll}\}}]$ 

```

data object corresponds to an s -dimensional attribute vector A_P , where $s = \sum_{i=1}^n s_i$. For example, as shown in Fig. 4, when $n = 3$, $s_i = 2 (1 \leq i \leq 3)$, and the attribute value of the spatial point P is the second value of the first attribute, the corresponding attribute vector is $A_P = (010000)$. In addition, if the Z code of the spatial point P is $Z_{\{P\}} = (011010)$, the plaintext index vector $I_P = (011010010000)$, which is then encrypted by $PE.Encrypt$.

When a conjunctive geometric range query is executed, the user generates a geometric vector Q_G according to the Z codes

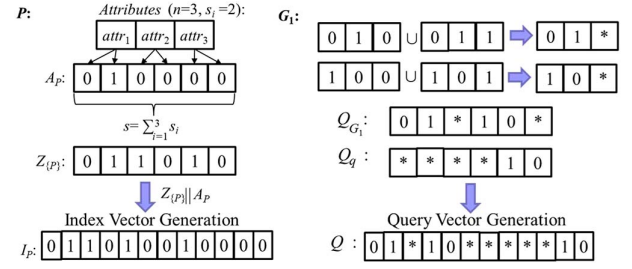


Fig. 4. An example of index and query vector generation.

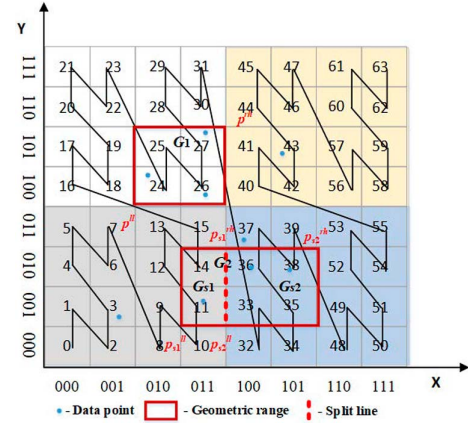


Fig. 5. An example of conjunctive geometric range query.

within the geometric range G , generates a query attribute vector Q_q according to the searched attributes, and then generates a query vector Q . As shown in Fig. 5, when a user wants to search the spatial objects with the first value of the third attribute in the geometric range query G_1 . He/she first converts the query G_1 into $Q_{G_1} = (010 \cup 011) || (100 \cup 101) = (01 * 10 *)$, and then generates the attribute vector $Q_q = (* ** * 10)$ to obtain the plaintext query vector $Q = Q_{G_1} || Q_q = (01 * 10 * ** * * 10)$. Finally, the query vector Q is encrypted with $PE.GenToken$. The query is the process of matching index vectors with query vectors through $PE.Query$.

In addition, we use the concept of spatial segmentation to avoid the unnecessary access of Z codes in the query process and support more efficient conjunctive geometric range queries. We deal with the spatial data in $\mathcal{P}_D = [0, x_0) \times \dots \times [0, x_{D-1}) \subseteq \mathcal{R}_D$. First, for $i \in [0, D - 1]$, the $\mathcal{P}_D$ is evenly divided into T_i partitions along each x_i -axis. The value of D in this manuscript is equal to 2 because the dimension of spatial data is reduced by using the Z-order space curve. Therefore, $\mathcal{P}_D$ is divided into T_x and T_y parts along the x and y axes respectively. The space $\mathcal{P}_D$ can be represented by $\mathcal{P}_D = \{\mathcal{P}_{D_1}, \mathcal{P}_{D_2}, \dots, \mathcal{P}_{D_\theta}\} | \theta = T_x \times T_y$. For a query $G_S \in \mathcal{R}_D$, it is easy to obtain the partitions intersected with G_S . Suppose these partitions are c_{i_0}, c_{i_1} , and $c_{i_{k-1}}$. Then, the G_S is further represented by $G_S = \bigcup_{j=0}^{k-1} G_{S_j}$ with $G_{S_j} = G_S \cap c_{i_j}$. When querying all the points in the G_S , the search algorithm only accesses k partitions and greatly reduces the search space.

C. Algorithm Description

The ECGRQ-LI scheme consists of four algorithms: *Setup*, *IndexBuild*, *TrapGen* and *Query*. The specific design details of the proposed scheme are described as follows:

- 1) *Setup*(1^λ) $\rightarrow$ *Key*: Given a security parameter λ , SDo runs *PE.Setup*(1^λ) to output a master secret key *MSK*, obtains a secret key *K* for the PRF function *F*, and lets *Key* = {*MSK*, *K*}.
- 2) *IndexBuild*(*Key*, *D*) $\rightarrow$ {*IP**, [*I*]}: SDo uses the secret key *MSK* and spatial data collection *D* to generate encrypted index vectors *IP** and an encrypted PM learned index [*I*].
 - Let the spatial data collection $D = \{D_1, \dots, D_i, \dots, D_m\}$ ($1 \leq i \leq m$), where $D_i = \{p_i, a_i\}$, p_i is a spatial data point, and $a_i \subseteq A_D$ is the attribute value of the spatial data object. For each $D_i = \{p_i, a_i\} \in D$, SDo calculates the Z code $Z_{\{p_i\}}$ of the spatial data point p_i , generates the attribute vector A_{p_i} according to a_i , and calculates $I_{p_i}^* \leftarrow PE.Encrypt(MSK, I_{p_i} = Z_{\{p_i\}} || A_{p_i}, \mu)$ to output $IP^* = \{I_{p_1}^*, \dots, I_{p_i}^*, \dots, I_{p_m}^*\}$.
 - Given the key *K*, for each point p_i ($1 \leq i \leq m$), SDo runs $F(K, Z_{\{p_i\}})$ to obtain the encrypted Z code $[Z_{\{p_i\}}]$. Then, the PM learning index [*i*] is constructed by learning the CDF distribution of $[Z_{\{p_i\}}]$ and uploaded to CSP.
- 3) *TrapGen*(*Key*, *G*, *q*) $\rightarrow$ *TQ*: SDu generates a trapdoor *TQ* for a query $\{G, q\}$ by the secret key *Key*, where *G* is the geometric range, and $q \in A_D$ is a query attribute set.
 - SDu converts the query *G* into vector Q_G according to the method mentioned in Section VI-B, and then uses *q* to generate query attribute vector Q_q , so as to obtain the plaintext query vector $Q = Q_G || Q_q$. It is worth noting that the query *G* does not need to be aligned with the grid, since the grid-based coverage method can be used to standardize any query range. Finally, SDu encrypts *Q* to generate the decryption key $SK \leftarrow PE.GenToken(MSK, Q)$.
 - To reduce access to unnecessary Z codes in the query process, we first partition the whole plane area $\mathcal{P}_D$ by the segmentation algorithm in Algorithm 2. Specifically, we divide the *x*-axis and *y*-axis into T_x and T_y parts respectively to obtain $\mathcal{P}_D = \{\mathcal{P}_{D_1}, \mathcal{P}_{D_2}, \dots, \mathcal{P}_{D_\theta}\}$, where $\theta = T_x \times T_y$.
 - For the geometric range G_S , if the point p^{rh} in the upper right corner and the point p^{ll} in the lower left corner of G_S are in the same partition, the encrypted Z codes $[Z_{\{p^{ll}\}}]$ and $[Z_{\{p^{rh}\}}]$ are calculated directly. Otherwise, we divide the query G_S into $\{G_{S_1}, \dots, G_{S_\gamma}\}$ ($1 \leq \gamma \leq \theta$) according to Algorithm 2 to ensure that the point p^{rh} in the upper right corner and the point p^{ll} in the lower left corner of each sub-query G_{S_i} ($1 \leq i \leq \gamma$) in G_S are in the same partition. For each sub-query

Algorithm 3 *Query*(*IP**, [*I*], *TQ*) $\rightarrow$ *RList*

Require: Encrypted index {*IP**, [*I*]}, trapdoor *TQ*;
Ensure: Query result list *RList*.

```

1: RList  $\leftarrow \phi$ 
2: Parses the TQ to obtain  $[Z_{\{p^{ll}\}}]$  and  $[Z_{\{p^{rh}\}}]$ 
3: for  $1 \leq i \leq \gamma$  do
4:    $Z_{start} \leftarrow [Z_{\{p_{S_i}^{ll}\}}]$ 
5:    $Z_{end} \leftarrow [Z_{\{p_{S_i}^{rh}\}}]$ 
6:    $Pos_{start} = Point\_search([I], Z_{start})$ 
7:    $Pos_{end} = Point\_search([I], Z_{end})$ 
8:   for  $s \leftarrow pos_{start}$  to  $pos_{end}$  do
9:      $flag_s \leftarrow PE.Query(Ip_s^*, SK)$ 
10:    if  $flag_s = 0$  then
11:       $RList \leftarrow RList + s(data)$ 
12:    end if
13:  end for
14:   $i = i + 1$ 
15: end for
16: return query result list RList

```

G_{S_i} , SDu computes the encrypted Z codes $[Z_{\{p_{S_i}^{ll}\}}]$ and $[Z_{\{p_{S_i}^{rh}\}}]$ of point $p_{S_i}^{ll}$ and point $p_{S_i}^{rh}$. Finally, let $[Z_{\{p^{ll}\}}] = [Z_{\{p_{S_1}^{ll}\}}], \dots, [Z_{\{p_{S_i}^{ll}\}}], \dots, [Z_{\{p_{S_\gamma}^{ll}\}}]$, $[Z_{\{p^{rh}\}}] = [Z_{\{p_{S_1}^{rh}\}}], \dots, [Z_{\{p_{S_i}^{rh}\}}], \dots, [Z_{\{p_{S_\gamma}^{rh}\}}]$. As shown in Fig. 5, $T_x = T_y = 2$. For query G_1 , query tokens $[Z_{\{p^{rh}\}}]$ and $[Z_{\{p^{rh}\}}]$ can be directly generated. For query G_2 , it is divided into two sub-queries G_{S_1} and G_{S_2} , and then $\{[Z_{\{p_{S_1}^{rh}\}}], [Z_{\{p_{S_2}^{rh}\}}]\}$ and $\{[Z_{\{p_{S_1}^{rh}\}}], [Z_{\{p_{S_2}^{rh}\}}]\}$ are calculated respectively.

- SDu generates the trapdoor $TQ = \{SK, [Z_{\{p^{ll}\}}], [Z_{\{p^{rh}\}}]\}$ and sends it to CSP.
- 4) *Query*(*IP**, [*I*], *TQ*) $\rightarrow$ *RList*: The query process is shown in Algorithm 3, the CSP takes as input the encrypted index {*IP**, [*I*]}, and trapdoor *TQ* and outputs a query result list *RList*.
 - After receiving the trapdoor *TQ* sent by the SDu, CSP parses the *TQ* to obtain $[Z_{\{p^{ll}\}}]$ and $[Z_{\{p^{rh}\}}]$. For each pair ($[Z_{\{p_{S_i}^{ll}\}}], [Z_{\{p_{S_i}^{rh}\}}]$) of $\{[Z_{\{p^{ll}\}}], [Z_{\{p^{rh}\}}]\}$, CSP uses the learned index [*i*] to execute point queries (6-7 lines) to narrow the search space of the conjunctive geometric range query.
 - CSP performs a query with Q^* on the query vectors within the geometric range. Specifically, for each Ip_i^* , CSP executes $flag \leftarrow PE.Query(Ip_i^*, SK)$. If $\exists flag = 1$, add Ip_i^* to the query result list *Rlist*. Finally, CSP returns the spatial data corresponding to *Rlist* to SDu.

VII. SECURITY ANALYSIS

In this section, we give the security analysis of the ECGRQ-LI. Since the predicate encryption [24] and pseudorandom function are the underlying structures of ECGRQ-LI, the IND-SCPA

security of ECGRQ-LI can be derived from the IND-SCPA of predicate encryptions and pseudorandom functions.

Theorem 1: As long as the predicate encryption is IND-SCPA secure, the ECGRQ-LI scheme is IND-SCPA secure.

Proof: We use the security game in the Fig. 2 to analyze the security of the ECGRQ-LI scheme. The idea is to illustrate that the security of ECGRQ-LI is equivalent to that of the PE algorithm.

Initialization: $\mathcal{C}$ runs $PE.Setup(1^\lambda)$ to obtain predicate encryption key MSK and generates the secret key K for pseudorandom function and keeps them private. The adversary $\mathcal{A}$ sends two plaintext index data sets $D_0 = \{D_{0,1}, \dots, D_{0,m}\}$ and $D_1 = \{D_{1,1}, \dots, D_{1,m}\}$ to $\mathcal{C}$, where $D_{0,i} = \{p_{0,i}, a_{0,i}\}$, $D_{1,i} = \{p_{1,i}, a_{1,i}\} (1 \leq i \leq m)$.

Requests: The adversary $\mathcal{A}$ adaptively selects multiple query requests and sends them to challenge $\mathcal{C}$. There are two types of requests.

-*Ciphertext:* For the j -th ciphertext request, $\mathcal{A}$ outputs D_j and sends it to $\mathcal{C}$, where $D_j = \{D_{j,1}, \dots, D_{j,i}, \dots, D_{j,m}\}$. For each $D_{j,i}$, $\mathcal{C}$ generates $Ip_{j,i} = \{Z_{\{p_{j,i}\}} || A_{p_{j,i}}\} (1 \leq i \leq n)$, and runs $Ip_{j,i}^* \leftarrow PE.Encrypt(MSK, Ip_{j,i}, \mu)$ and $[Z_{\{p\}}] \leftarrow F(K, Z_{\{p\}})$, and obtains the learned index $[I_j]$ by learning the data distribution of $[Z_{\{p\}}]$. Finally, the challenger $\mathcal{C}$ responds with encrypted index vectors $IP_j^* = \{I_{p_{j,1}}^*, \dots, I_{p_{j,i}}^*, \dots, I_{p_{j,n}}^*\}$ and $[I_j]$.

-*Trapdoor:* For the j -th trapdoor request, $\mathcal{A}$ outputs the conjunctive geometric range query $Q_j = \{G_j, q_j\}$, where G_j and q_j can be expressed as $\{(p_j^{rh}, p_j^{ll}), q_j\}$. The challenger $\mathcal{C}$ encrypts Q_j to obtain the query trapdoor $SK_j \leftarrow PE.GenToken(MSK, Q_j)$. In addition, the challenger $\mathcal{C}$ encrypts (p_j^{rh}, p_j^{ll}) to obtain $([Z_{\{p_j^{ll}\}}], [Z_{\{p_j^{rh}\}}]) \leftarrow F(K, (p_j^{rh}, p_j^{ll}))$ and takes $TQ_j = \{SK_j, [Z_{\{p_j^{ll}\}}], [Z_{\{p_j^{rh}\}}]\}$ as the response. Moreover, the query needs to meet the following conditions. $D_{0,i} \in (p_j^{rh}, p_j^{ll}) \wedge D_{1,i} \in (p_j^{rh}, p_j^{ll})$ or $D_{0,i} \notin (p_j^{rh}, p_j^{ll}) \wedge D_{1,i} \notin (p_j^{rh}, p_j^{ll})$. The above is equivalent to the constraint $\mathcal{L}(D_0, Q_j) = \mathcal{L}(D_1, Q_j)$ in Fig. 2.

Challenge: For D_0 and D_1 selected in *initialization*, the challenger $\mathcal{C}$ randomly selects $b \in \{0, 1\}$ and returns $DB = (D_{b,1}, \dots, D_{b,i}, \dots, D_{b,n})$ to the adversary $\mathcal{A}$. $\mathcal{A}$ continues to send multiple queries adaptively.

Guess: The adversary $\mathcal{A}$ guesses that b is b' .

Therefore, the index security problem of ECGRQ-LI is successfully simplified to the data security problem of the predicate encryption and the probability that the adversary $\mathcal{A}$ can distinguish D_0 and D_1 is

$$\begin{aligned} & Adv_{(ECGRQ-LI)}^{IND-SCPA-Index}(1^\lambda) \\ &= 1 - ((1 - Adv_{(PE)}^{Index}(1^\lambda))(1 - Adv_{(PRF)}^{Index}(1^\lambda)))^m \\ &\leq negl(\lambda). \end{aligned} \quad (4)$$

As described in Theorem 1, ECGRQ-LI is index secure as long as the predicate encryption is IND-SCPA secure. Similarly, since trapdoors are generated by the predicate encryption we give Theorem 2.

Theorem 2: As long as the predicate encryption and pseudorandom function are IND-SCPA secure, the ECGRQ-LI scheme is IND-SCPA query privacy.

Proof: Similar to Theorem 1, the index security problem of ECGRQ-LI can be simplified to the data security problem of the predicate encryption and pseudorandom function, and the IND-SCPA query privacy can be proved in a similar way by using the definition 5.

Initialization: $\mathcal{C}$ runs $PE.Setup(1^\lambda)$ to obtain predicate encryption key MSK and generates the secret key K for pseudorandom function and keeps them private. The adversary $\mathcal{A}$ sends two plaintext index queries $Q_0 = \{G_0, q_0\}$ and $Q_1 = \{G_1, q_1\}$ to $\mathcal{C}$, where G_0 and G_1 are geometric ranges, $q_0 \in A_p$ and $q_1 \in A_p$ are query attribute sets.

Requests: $\mathcal{A}$ adaptively selects multiple query requests and sends them to $\mathcal{C}$. There are two types of requests.

-*Ciphertext:* For the j -th ciphertext request, $\mathcal{A}$ outputs D_j and sends it to $\mathcal{C}$, where $D_j = \{D_{j,1}, \dots, D_{j,i}, \dots, D_{j,m}\}$. For each $D_{j,i}$, $\mathcal{C}$ generates $Ip_{j,i} = \{Z_{\{p_{j,i}\}} || A_{p_{j,i}}\} (1 \leq i \leq n)$, and runs $Ip_{j,i}^* \leftarrow PE.Encrypt(MSK, Ip_{j,i}, \mu)$ and $[Z_{\{p\}}] \leftarrow F(K, Z_{\{p\}})$, and obtains the learned index $[I_j]$ by learning the data distribution of $[Z_{\{p\}}]$. Finally, the challenger $\mathcal{C}$ responds with encrypted index vectors $IP_j^* = \{I_{p_{j,1}}^*, \dots, I_{p_{j,i}}^*, \dots, I_{p_{j,n}}^*\}$ and $[I_j]$. Moreover, the query needs to meet the following conditions. 1) $\mathcal{L}(IP_j^*, Q_0) = \mathcal{L}(IP_j^*, Q_1)$. 2) $IP_j^* \in G_0 \wedge IP_j^* \in G_1$ or $IP_j^* \notin G_0 \wedge IP_j^* \notin G_1$.

-*Trapdoor:* For the j -th trapdoor request, $\mathcal{A}$ outputs the conjunctive geometric range query $Q_j = \{G_j, q_j\}$, where G_j and q_j can be expressed as $\{(p_j^{rh}, p_j^{ll}), q_j\}$. The challenger $\mathcal{C}$ encrypts Q_j to obtain the query trapdoor $SK_j \leftarrow PE.GenToken(MSK, Q_j)$. In addition, the challenger $\mathcal{C}$ encrypts (p_j^{rh}, p_j^{ll}) to obtain $([Z_{\{p_j^{ll}\}}], [Z_{\{p_j^{rh}\}}]) \leftarrow F(K, (p_j^{rh}, p_j^{ll}))$ and takes $TQ_j = \{SK_j, [Z_{\{p_j^{ll}\}}], [Z_{\{p_j^{rh}\}}]\}$ as the response.

Challenge: For Q_0 and Q_1 selected in *initialization*, the challenger $\mathcal{C}$ randomly selects $b \in \{0, 1\}$ and returns $TQ_b = (SK_b, [Z_{\{p_b^{ll}\}}], [Z_{\{p_b^{rh}\}}])$ to the adversary $\mathcal{A}$. $\mathcal{A}$ continues to send multiple queries adaptively.

Guess: The adversary $\mathcal{A}$ guesses that b is b' .

The query privacy problem of ECGRQ-LI is successfully simplified to the security problem of the predicate encryption and the probability that $\mathcal{A}$ can distinguish Q_0 and Q_1 is

$$\begin{aligned} & Adv_{(ECGRQ-LI)}^{IND-SCPA-Query}(1^\lambda) \\ &= 1 - ((1 - Adv_{(PE)}^{Query}(1^\lambda))(1 - Adv_{(PRF)}^{Query}(1^\lambda)))^{n_Q} \\ &\leq negl(\lambda), \end{aligned} \quad (5)$$

where n_Q represents the number of query requests sent by $\mathcal{A}$.

Discussion.

1) *Reconstruction attacks.* We discuss a specific reconstruction attack [21] against the leakage of access patterns to further elaborate its security. Attackers depend on the observed $RS(D)$ and $RM(D)$ sets of the database D to recover the user's private data. $RM(D)$ is the response multiset of a database D , i.e., the multiset of all access patterns of D . The set $RS(D)$ is obtained by removing any duplicate entries from $RM(D)$. Subsequently, the attacker i) finds the extreme points of database D by using the difference between the largest response set and the second largest response set in $RS(D)$; ii) extracts the

extreme points on the left and right and divides the remaining points into three sets; iii) finds all possible positions of left and right extreme points, and use these points to identify one or two positions of each point of D ; iv) uses the recovered locations to divide database D into components; v) reduces the position of each point in the component to one. Such that the attacker obtains the set of databases equivalent to D in time $O(N_0 + N_1)(RN^2 + R\log R)$.

In our scheme, the responses are different for the same queries since the data user can hide access patterns by differential privacy technology and adjust the value of ε to balance security and query performance. Attackers cannot judge whether the search results come from the same queries through analyzing access patterns and the first step of the attack cannot be performed. Therefore, ECGRQ-LI can defend against the reconstruction attack.

2) *Scalability*. In practical application scenarios, dynamic updates of spatial data (such as vehicle data) are important because there may be an increase in data volume or changes in data distribution. We can introduce dynamic learned indexing techniques [35] to solve the problem of updating encrypted spatial data, where [35] is able to support efficient insertion and deletion of spatial data by generating searchable data layouts on disk pages using learned models. In addition, updating operations may leak more spatial data information (such as the correlation between newly inserted data and previous search tokens), making the scheme vulnerable to file injection attacks. For this situation, we can combine an oblivious data structure [31] to ensure the forward and backward security of the dynamic learning index during the update process. Forward security ensures that the solution does not leak information about newly inserted files that match previous search queries during the update period. Backward security ensures that the information of previously inserted and subsequently deleted files is not leaked during two queries for the same keyword.

VIII. PERFORMANCE ANALYSIS

A. Theoretical Analysis

In this section, we theoretically analyze the performance of the proposed ECGRQ-LI scheme and compare it with existing schemes in terms of index construction cost, trapdoor generation cost and query cost.

Index construction cost. For the *IndexBuild* algorithm of ECGRQ-LI, it first encrypts each data vector $I_{p_i} (1 \leq i \leq m)$ with the predicate encryption algorithm (i.e., SHVE) to obtain $IP^* = \{I_{p_1}^*, \dots, I_{p_i}^*, \dots, I_{p_m}^*\}$. In addition, ECGRQ-LI learns the data distribution of Z codes corresponding to the above data vectors to train the PM learned index and reduce the search space. Therefore, the index generation time overhead of ECGRQ-LI is $O(m)(T_{Enc} + T_{PM})$, where T_{Enc} is the execution time of predicate encryption, m is the database size, and T_{PM} is the generation time of the PM learned index.

Trapdoor generation cost. For the *TrapGen* algorithm of ECGRQ-LI, it connects the vector Q_G and the vector Q_q to obtain the plaintext query vector Q , encrypts the query vector Q with the *GenToken* algorithm of predicate encryption to obtain

SK , and reduces the search space by generating $([Z_{p^u}], [Z_{p^r h}])$ and using the PM index. Therefore, the cost of the trapdoor generation is $T_{Gen} + 2\gamma T_{PRF}$, where T_{Gen} is the execution time of trapdoor generation algorithm for predicate encryption, γ is the number of sub-queries.

Query cost. For the *Query* algorithm of ECGRQ-LI, CSP uses the PM learned index to estimate the location range to search. For each index vector $I_{p_i}^*$ in the range, CSP performs $PE.Query(I_{p_i}^*, SK)$ to obtain the final query results. Therefore, the query cost of ECGRQ-LI is $O(\kappa)T_{Dec}$, κ is the number of Z codes within the location range, T_{Dec} is the execution time of the predicate encryption query algorithm.

Table III shows the comparison results with existing schemes. It can be seen that our scheme has great advantages in query efficiency and storage overhead. Specifically, GRSE-tree [8] and FastGeo [9] are proposed based on the SSW encryption [23]. PBRQ-T and PBRQ-T⁺ in [14] are proposed using Katz-Sahai-Waters (KSW) encryption that is an asymmetric predicate encryption. ECGRQ-LI mainly uses the symmetric predicate encryption and pseudo-random function to avoid expensive pairing operations and exponentiation operations. Therefore, the efficiency of our scheme is significantly higher than the above two schemes. In addition, due to the use of traditional index structures (i.e. R-tree [8], [10], [13], Quadtree [14] and Binary tree [12]), the corresponding schemes search for appropriate child nodes through a large number of non-leaf nodes. ECGRQ-LI uses the learned index PM instead of the traditional index tree structure to execute queries. Since there is no search process between each stage of the learned index, the query cost is independent of the database size m . Therefore, the query cost of the above schemes is higher than that of the ECGRQ-LI scheme. Similarly, the use of traditional tree indexes also leads to the storage overhead of these schemes being higher than that of ECGRQ-LI. Furthermore, compared with the existing schemes, only our scheme supports conjunctive geometric range queries of n -dimensional spatial data and protects the access pattern of data users.

B. Experimental Evaluations

Implementation. We implement ECGRQ-LI and existing schemes with Java to compare the performance. For the ECGRQ-LI scheme, the dimensionality reduction preprocessing of spatial data is realized by using the code of the Z-order curve³. The learned model is based on the code of the RIM model [16]. The difference is that an exponential query is used to locate the correct positions to improve efficiency. In addition, we use the JPBC library to implement the pairing operations of GRSE-tree [8] and FastGeo [9], which is evaluated on the curve $y^2 = x^3 + x$. We implemented PRF evaluations in [9], [11], [12] with SHA-256. According to [8], the height of the BQ-tree in the PBRQ-T⁺ scheme [14] and the R-tree in the GRSE-tree scheme [8] is set to 16 for fairness. All the experiments are performed on a computer with an Intel i7-6300U processor and the Ubuntu 20.04 operation system. The indicators are the

³<https://github.com/eren-ck/MortonLib>

TABLE III a
COMPARISON WITH PREVIOUS GRQ AND CGRQ SCHEMES

Schemes	GRSE-tree [8]	FastGeo [9]	EGRQ [10]	I [12]	II [12]
Index	$O(m + m_o)$ $(6d_2 + 2)T_e$	$O(\tau)((6T + 2)T_e + T_{PRF})$	$O(m)T_{KNN.Enc} + O(m_0)T_{PRF} + T_R$	$O(2^t m + 2(2^H - 1)(T_{ASHE.Enc} + T_{PRF}))$	$O(2^t m + 2(2^H - 1)(T_{ASHE.Enc} + T_{PRF}))$
Trapdoor	$8d_2 T_e$	$O(\tau_0)(8T_e + T_{PRF})$	$T_{KNN.Gen} + T_{PRF}$	$O(2(2^H - 1) + \log(R)T_{PRF})$	$O(2(2^H - 1) + \log(R)T_{PRF})$
Query	$O(\log_m)(2d_2 + 2)T_p$	$O(\tau_0)(2T + 2)T_p$	$O(K \log_h m)T_{Que}$	$O(m) \log(2R)(T_{ASHE.Dec} + T_{PRF})$	$O(m) \log(2R)(T_{ASHE.Dec} + T_{PRF})$
SO	$O(d_2(m + m_o))$	$O(\tau m)$	$O((N + K + T)(m + m_o))$	$2^H m$	$2^H m$

TABLE III b
COMPARISON WITH PREVIOUS GRQ AND CGRQ SCHEMES

Schemes	PBRQ-T [14]	PBRQ-T ⁺ [14]	[13]	Ours
Index	$O(m)(4d_2 T_e + T_{tree})$	$O(m_0)(20d_1 + 4d_2)T_e + O(m)(4d_2 T_e + T_{tree})$	$O(m)2T_{HE.ENC}$	$O(m)(T_{Enc} + T_{PRF}) + T_{PM}$
Trapdoor	$(4d_1 k + 4d_2 + 2k)T_e + T_S$	$(4d_1 k + 4d_2 + 2k)T_e + T_S$	$3T_{HE.ENC}$	$T_{Gen} + 2\gamma T_{PRF}$
Query	$O(m)(2M + 2d_1 k + 2d_2)T_p$	$(\log_2 L)(2d_1 k + 2d_2)T_p + 2(d_2 + M)m'T_p$	$O(m)(L + K \log_2 m)(T_{HE.ENC} + T_e)$	$O(\kappa)T_{Dec}$
SO	$O(m)((1 + 2d) + S_\Gamma)$	$O(m_0)(10d_1 + 2d_2) + O(m)(1 + 2d_2)S_\Gamma$	$O(m)(12(\mu + 2))$	$O(m)(s + t)$

In the table, the notation m_0 is the number of non-leaf nodes in the index tree. τ is the total number of spatial vector instances in the database; τ_0 is the number of index instances that the server needs to evaluate; k is the number of range tokens; K is the number of returned results; T_p represents the time required to calculate the pairing operation; T_e is the time to calculate the exponentiation operation; T_{PRF} is the time taken to execute the pseudo-random function; T_{Enc} is the time to perform symmetric encryption of SHVE [24]; T_{Dec} is the time taken to perform symmetric decryption of SHVE [24]; N represents the highest degree of the term in the fitted polynomial in [10]; K represent the coefficients of the role polynomial; l is the bit length of database objects; T are the extended dimensions; H is the height of the binary tree; $T_{HE.ENC}$ is the encryption time of the Paillier algorithm; t is the length of the Z code; s is the length of the attribute query vector; M is the number of non-leaf node in the access tree in [14]; S is the user attribute set [14]; S_Γ be the storage costs of an access tree. $T_{tree} = (2 + 6|N_t| + 2|N_{nt}|)$; $T_S = 6|S_t|T_e + 3|S_{nt}|T_e$; $d = d_1 + d_2$ represents the size of the bitmap in [14]; SO is the short for “Storage Overhead”.

running time of algorithms, where the average value of 30 independent runs of an algorithm is taken as the experimental result.

Dataset. We use the Geolife GPS trajectory⁴ and GeoNames⁵ datasets to demonstrate the performance of the scheme. The Geolife GPS trajectory dataset consists of the GPS trajectories of 182 users, consisting of a total of 24,876,978 data points. The GeoNames dataset contains 11,540,325 geographical locations. Each data point in both datasets is represented using latitude and longitude. This paper randomly selects 2,485,408 and 1,153,580 data points from the Geolife GPS trajectory and GeoNames datasets, and maps the data points to the graph. The specific distribution of data points is shown in Fig. 6, where the horizontal axis represents latitude and the vertical axis represents longitude. Since the objects in the Geolife GPS trajectory and GeoNames datasets do not contain text keywords, this paper also uses Enron dataset⁶ to extract attribute values for each spatial data object.

Index model training. We first test the performance of the PM index structure. After debugging, the model with a hidden layer has better test results. Therefore, a hidden layer model that has 128 neurons is used in the experiments and a ReLU layer is connected for nonlinear transformation. The output layer represents the corresponding position of the spatial data. The learning rate is 0.0001 and the batch training size is 1000. As shown in Fig. 7(a), the learned models for the Geolife GPS



Fig. 6. The specific distribution of data points.

trajectory and GeoNames datasets finally converge to a fixed loss value with the training process, which fits the distribution of the data. Moreover, Fig. 7(b) shows that the training process is related to the dataset size m . The larger is the value of m , the slower the convergence speed of the model. The reason is that the model can learn faster on a small data scale.

We further test the performance of executing point queries on the PM index. For the Geolife GPS trajectory and GeoNames datasets, the effect of training rounds and privacy budgets on the point query accuracy is shown in Fig. 8. With the increase of the number of training rounds, the query accuracy of the model with different noise increases gradually and tends to converge after 40 rounds. In addition, the smaller the value of ε , the more noise added. Therefore, with the increase of ε , the degree of privacy protection becomes stronger. On the contrary, the accuracy of the point query is reduced. Users can balance security and query performance by adjusting the value of ε .

⁴<https://www.microsoft.com/en-us/research/publication/geolife-gps-trajectory-dataset-user-guide/>

⁵<https://www.kaggle.com/datasets/geonames/geonames-database>

⁶<http://www.cs.cmu.edu/~enron/>

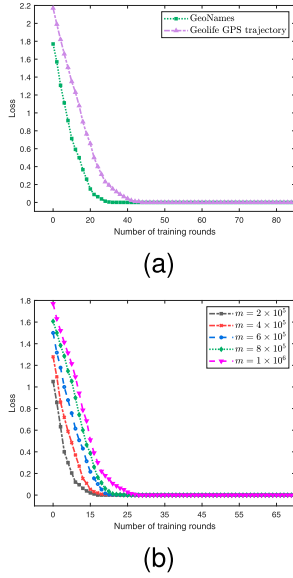


Fig. 7. Model training process.

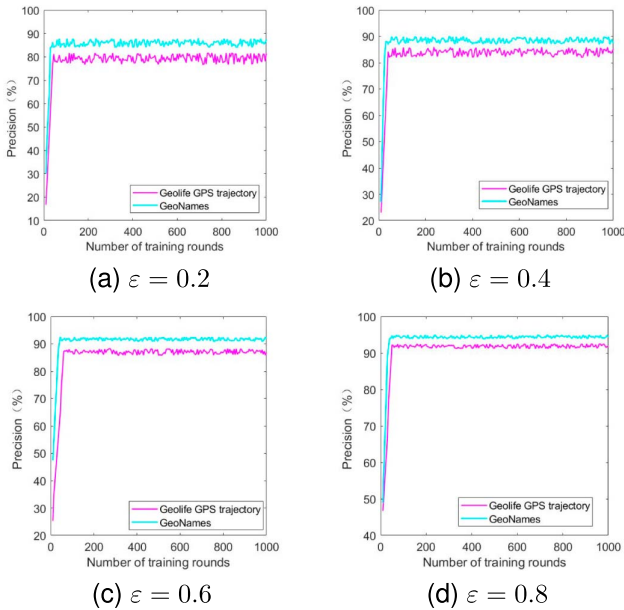


Fig. 8. The impact of the number of training rounds on the query accuracy.

Segmentation algorithm evaluation. Let the value of ϵ be 0.4. In this case, the values of $\min_{error}$ and $\max_{error}$ are 96 and 73, respectively. We then test the query performance of the ECGRQ-LI scheme and compare it with the previous schemes. Before displaying the comparison results, it is necessary to verify the effectiveness of spatial segmentation and query segmentation. When the query region is a 64×64 square, we debug the ECGRQ-LI scheme on different datasets to obtain the appropriate number of partitions. For the datasets of different sizes, the number of partitions corresponding to the optimal query performance is different. In addition, the query time of ECGRQ-LI does not always decrease with the increase of the

number of partitions. The reason is that a large number of partitions can lead to an increase in the number of sub-queries γ , that is, the number of visits to the learned index PM increases.

Therefore, we provide users with a method to determine the number of partitions. As shown in formula (5), when the number of partitions increases, the number of corresponding sub-queries changes from γ to γ' , and the number of Z codes accessed changes from $\sum_{G_{S_i} | 1 \leq i \leq \gamma \subseteq G_S} (Z_{\{p_{S_i}^{rh}\}} - Z_{\{p_{S_i}^{ll}\}})$ to $\sum_{G_{S_i} | 1 \leq i \leq \gamma' \subseteq G_S} (Z_{\{p_{S_i}^{rh}\}} - Z_{\{p_{S_i}^{ll}\}})$. We can use their ratio R_P to evaluate the impact of the number of partitions on efficiency.

$$R_P = \frac{(\gamma' - \gamma) \sum_{G_{S_i} | 1 \leq i \leq \gamma' \subseteq G_S} (Z_{\{p_{S_i}^{rh}\}} - Z_{\{p_{S_i}^{ll}\}})}{\sum_{G_{S_i} | 1 \leq i \leq \gamma \subseteq G_S} (Z_{\{p_{S_i}^{rh}\}} - Z_{\{p_{S_i}^{ll}\}})} \quad (6)$$

Search. We compare the ECGRQ-LI scheme with previous schemes on our datasets. In addition, we test the impact on query time when the dataset size m changes from 2×10^5 to 1×10^6 . The size of the bloom filter of the GRSE-tree scheme is set as 1×10^4 . The query region is a 64×64 square and there are three attribute values. As shown in Table IV, the ECGRQ-LI scheme is much more efficient than previous schemes. In particular, due to the use of the learned index and SHVE, the query efficiency of ECGRQ-LI is faster than the state-of-the-art scheme PBRQ-T⁺ scheme supporting conjunctive geometric range queries in [14] $32 \times$ to $47 \times$. Moreover, we also remove the attribute vector in ECGRQ-LI and compare it with the existing geometric range query schemes [8], [9], [12] whose index vectors do not contain attribute vectors and fails to perform conjunctive keyword queries. As shown in Table IV, the query time of FastGeo [9] is at least $300 \times$ that of the ECGRQ-LI scheme since FastGeo needs to perform a lot of pairing and exponentiation operations. In addition, the efficiency of ECGRQ-LI is $2.1 \times$ to $3.3 \times$ faster than [12].

Index construction. We compare the index construction time with previous schemes in Fig. 9. As can be seen in Fig. 9(a), the index construction time of the three schemes increases with the increase of m . This is because the larger m is, the longer the index vectors will be, which affects the generation time of encrypted index vectors. Fig. 9(b) shows that the index building time also increases linearly with the size of the attribute set, where $m = 1 \times 10^6$. The reason is that the number of bits of each attribute query vector increases with the increase of s , affecting the generation time of encrypted index vectors. In addition, the running time of ECGRQ-LI is longer than that of PBRQ-T and PBRQ-T⁺, which is caused by the additional training time of the learned index model. However, it does not affect the application of ECGRQ-LI, since data users pay more attention to the response time of the queries and the extra construction time than PBRQ-T and PBRQ-T⁺ is tolerable. As shown in Fig. 9(c), the learned index construction time for different training rounds is also compared with existing schemes. It can be seen that the larger the value of ϵ , the smaller the number of training rounds and the index construction time decreases. It is worth noting that our index construction time is only compared with [14]. The reason is that only [14] implements conjunctive geometric range queries of multi-dimensional spatial data like our ECGRQ-LI scheme, thus the index vector

TABLE IV
QUERY PERFORMANCE COMPARISON

Schemes	$m = 2 \times 10^5$	$m = 4 \times 10^5$	$m = 6 \times 10^5$	$m = 8 \times 10^5$	$m = 1 \times 10^6$	GeoNames	Geolife GPS trajectory
GRSE-tree [8]	2.77h	5.32h	8.29h	10.82h	15.96h	17.52h	30.76h
FastGeo [9]	552.33s	833.62s	1138.16s	1364.28s	1662.21s	1953.17s	2762.89s
[13]	92.05s	135.94s	162.59s	194.89s	207.77s	244.14s	306.98s
I [12]	3.15s	5.35s	8.03s	10.71s	12.96s	13.51s	19.96s
II [12]	3.55s	6.74s	9.77s	11.59s	13.87s	15.57s	21.75s
ECGRQ-LI*	1.62s	2.19s	3.32s	3.97s	4.16s	5.77s	7.16s
PBRQ-T [14]	806.4s	1265.1s	1363.3s	1792.1s	2509.3s	29381.1s	3609.2s
PBRQ-T ⁺ [14]	65.2s	108.4s	120.7s	179.2s	229.5s	360.2s	433.9s
ECGRQ-LI	2.26s	3.41s	3.67s	4.59s	5.17s	7.59s	10.17s

ECGRQ-LI* represents the scheme after removing the attribute vectors in scheme ECGRQ-LI.

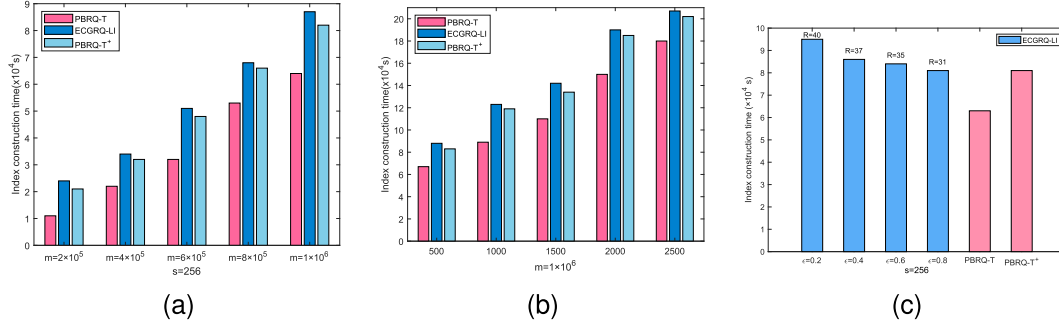


Fig. 9. Comparison of index construction time.

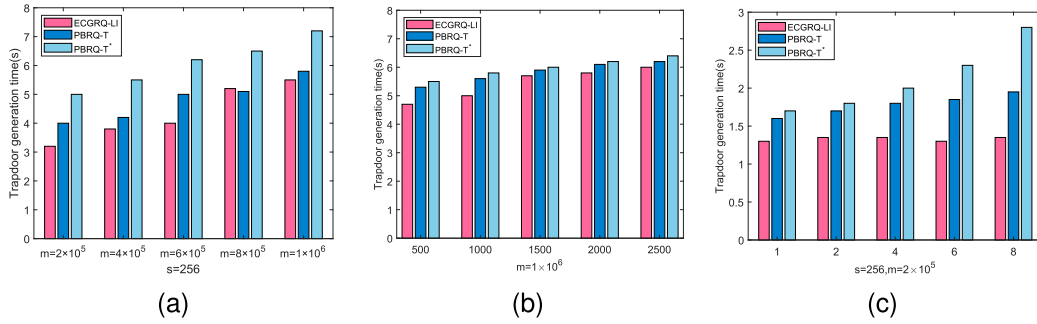


Fig. 10. Comparison of trapdoor generation time.

structures of the schemes are consistent and comparable. Similarly, due to the involvement of query vector generation, the trapdoor generation time is only compared with [14].

Index memory size. We compare the index memory size of ECGRQ-LI with the PBRQ-T⁺ scheme [14] and the II scheme [12] that uses the traditional Quadtree and Binary tree to construct search indexes respectively. The experimental results show that for $m = 10^6$, the storage overhead of our scheme is 56.3MB and that of the PBRQ-T⁺ scheme and the II scheme are 165.5MB and 137.3MB. That is, ECGRQ-LI uses 66% and 59% less memory than the PBRQ-T⁺ scheme and the II scheme to store encrypted index data. This is because the neural network only includes several parameters in the learned index model, which can save the index memory size.

Trapdoor generation. We compare the trapdoor generation time with the previous schemes in Fig. 10. Obviously, our

scheme is more efficient than PBRQ-T and PBRQ-T⁺ because they have many exponential and pairing operations in the process of generating trapdoors. Moreover, as shown in Fig. 10(a) and 10(b), like the index building process, the trapdoor generation time increases with the increase of m and s . This is because the number and length of query vectors will increase as the values of m and s increase, which raises the computational cost of *GenToken* algorithm. In addition, the trapdoor generation time is related to the value of k which is the number of range tokens in [14] and denotes the number of sub-queries in our scheme. Therefore, we test the trapdoor generation time of the ECGRQ-LI and the schemes in [11] when the values of m and s are fixed in Fig. 10(c). It can be seen that the trapdoor generation time of the three schemes increases with the increase of the value of k . However, the growth rate of *TrapGen* running time of the ECGRQ-LI scheme is slow. This is because the increase

of the k value will not increase the length of the query vectors and the predicate encryption time.

IX. CONCLUSION

To achieve efficient conjunctive geometric range queries of encrypted spatial data, ECGRQ-LI is proposed in this paper. The novelty is that the ECGRQ-LI scheme uses the proposed privacy-preserving learned index to reduce the search space and storage overhead. In particular, ECGRQ-LI expands the objective function by polynomials and adds noise to each coefficient, which can protect data privacy by interfering with the output results of the learned index. In addition, the concept of spatial segmentation is introduced to reduce the access of Z codes in the query process. A large number of simulation results show that the search performance of the ECGRQ-LI scheme is much more efficient than previous schemes, and the index storage overhead is significantly reduced. The security analysis shows that ECGRQ-LI satisfies the IND-SCPA security, which can ensure data security and query privacy in the query process.

In the future, we will expand the practicality of the proposed scheme by applying it to vehicle social networks. This is because the development of wireless sensor networks has made it easy to collect vehicle data from vehicles or roadside units, which can provide convenient social services (e.g. improving the efficiency of traffic police in handling traffic accidents). However, vehicle data typically contains a lot of sensitive information, such as location information. Therefore, it is necessary to design a query scheme for encrypting vehicle location data in vehicle social networks. Moreover, to solve the problem of long index construction time caused by the additional training time required for learned index models, we plan to reduce the training time by optimizing the training process and using distributed training methods.

REFERENCES

- [1] Q. Jiang et al., "Reducing paging and exit overheads in intel SGX for oblivious conjunctive keyword search," *IEEE Trans. Comput.*, vol. 74, no. 3, pp. 776–789, Mar. 2025.
- [2] S. Hu, C. Cai, Q. Wang, C. Wang, Z. Wang, and D. Ye, "Augmenting encrypted search: A decentralized service realization with enforced execution," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 6, pp. 2569–2581, Nov./Dec. 2021.
- [3] W. Yang and Y. Zhu, "A verifiable semantic searching scheme by optimal matching over encrypted data in public cloud," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 100–115, 2021.
- [4] Y. Miao, R. H. Deng, X. Liu, K.-K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 4, pp. 1667–1680, Jul./Aug. 2021.
- [5] L. Xu, H. Duan, A. Zhou, X. Yuan, and C. Wang, "Interpreting and mitigating leakage-abuse attacks in searchable symmetric encryption," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 5310–5325, 2021.
- [6] H. Zhu, R. Lu, C. Huang, L. Chen, and H. Li, "An efficient privacy-preserving location-based services query scheme in outsourced cloud," *IEEE Trans. Veh. Technol.*, vol. 65, no. 9, pp. 7729–7739, Sep. 2016.
- [7] B. Wang, M. Li, H. Wang, and H. Li, "Circular range search on encrypted spatial data," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Piscataway, NJ, USA: IEEE Press, 2015, pp. 182–190.
- [8] B. Wang, M. Li, and H. Wang, "Geometric range search on encrypted spatial data," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 4, pp. 704–719, Apr. 2016.
- [9] B. Wang, M. Li, and L. Xiong, "FastGeo: Efficient geometric range queries on encrypted spatial data," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 2, pp. 245–258, Mar./Apr. 2019.
- [10] G. Xu, H. Li, Y. Dai, K. Yang, and X. Lin, "Enabling efficient and geometric range query with access control over encrypted spatial data," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 870–885, Apr. 2019.
- [11] X. Wang et al., "Search me in the dark: Privacy-preserving Boolean range query over encrypted spatial data," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 2253–2262.
- [12] S. K. Kermanshahi et al., "Geometric range search on encrypted data with forward/backward security," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 698–716, Jan./Feb. 2022.
- [13] Y. Miao, L. Song, X. Li, H. Li, K. R. Choo, and R. H. Deng, "Privacy-preserving arbitrary geometric range query in mobile internet of vehicles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7725–7738, Jul. 2024.
- [14] Q. Tong, X. Li, Y. Miao, X. Liu, J. Weng, and R. Deng, "Privacy-preserving Boolean range query with temporal access control in mobile computing," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 5159–5172, May 2023.
- [15] Z. Shang, S. Oya, A. Peter, and F. Kerschbaum, "Obfuscated access and search patterns in searchable encryption," 2021, *arXiv:2102.09651*.
- [16] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proc. Int. Conf. Manage. Data*, 2018, pp. 489–504.
- [17] J. Ding et al., "Alex: An updatable adaptive learned index," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 969–984.
- [18] X. Li, J. Li, and X. Wang, "ASLM: Adaptive single layer model for learned index," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, New York, NY, USA: Springer-Verlag, 2019, pp. 80–95.
- [19] P. Ferragina and G. Vinciguerra, "The PGM-index: A fully-dynamic compressed learned index with provable worst-case bounds," *Proc. VLDB Endowment*, vol. 13, no. 8, pp. 1162–1175, 2020.
- [20] H. Wang, X. Fu, J. Xu, and H. Lu, "Learned index for spatial queries," in *Proc. 20th IEEE Int. Conf. Mobile Data Manage. (MDM)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 569–574.
- [21] F. Falzon, E. A. Markatou, D. Cash, A. Rivkin, J. Stern, and R. Tamassia, "Full database reconstruction in two dimensions," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 443–460.
- [22] W. Lin, K. Wang, Z. Zhang, and H. Chen, "Revisiting security risks of asymmetric scalar product preserving encryption and its variants," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Piscataway, NJ, USA: IEEE Press, 2017, pp. 1116–1125.
- [23] E. Shen, E. Shi, and B. Waters, "Predicate privacy in encryption systems," in *Proc. Theory Cryptography Conf.*, New York, NY, USA: Springer-Verlag, 2009, pp. 457–473.
- [24] S. Lai et al., "Result pattern hiding searchable encryption for conjunctive queries," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 745–762.
- [25] G. Yang, L. Liang, A. Hadian, and T. Heinis, "FLIRT: A fast learned index for rolling time frames," in *Proc. EDBT*, 2023, pp. 234–246.
- [26] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proc. IEEE Symp. Secur. Privacy*, 2002, pp. 44–55.
- [27] X. Yang et al., "Multi-client verifiable encrypted keyword search scheme with authorization over outsourced encrypted data," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 6356–6371, Nov./Dec. 2024.
- [28] X. Zhao, X. Yang, J. Wang, and S. Qi, "Obric: Oblivious keyword search in hyperledger fabric platform," in *Proc. Int. Conf. Blockchain Trustworthy Syst.*, New York, NY, USA: Springer-Verlag, 2023, pp. 176–191.
- [29] X. Yang, A. Wang, Q. Wang, S. Qi, and Y. Qi, "A practical and privacy-preserving vehicular data sharing framework by using blockchain," in *Proc. IEEE 22nd Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1300–1305.
- [30] G. Chen, T.-H. Lai, M. K. Reiter, and Y. Zhang, "Differentially private access patterns for searchable symmetric encryption," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 810–818.
- [31] M. Li, C. Jia, R. Du, W. Shao, and G. Ha, "DSE-RB: A privacy-preserving dynamic searchable encryption framework on redactable blockchain," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2856–2872, Jul./Sep. 2023.

- [32] E. Shen, E. Shi, and B. Waters, "Predicate privacy in encryption systems," in *Theory of Cryptography*. Berlin, Germany: Springer Berlin Heidelberg, 2009, pp. 457–473.
- [33] R. Guo, B. Qin, Y. Wu, R. Liu, H. Chen, and C. Li, "LuxGeo: Efficient and secure enhanced geometric range queries," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1775–1790, Feb. 2023.
- [34] Q. Song, Z. Liu, J. Cao, K. Sun, Q. Li, and C. Wang, "SAP-SSE: Protecting search patterns and access patterns in searchable symmetric encryption," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1795–1809, 2021.
- [35] P. Li, H. Lu, Q. Zheng, L. Yang, and G. Pan, "LISA: A learned index structure for spatial data," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 2119–2133.



Mingyue Li received the M.Eng. degree from Hebei University, China, in 2019, and the Ph.D. degree from Nankai University, China, in 2022. In 2022, she joined Hebei University, China, where she is currently a Lecturer with the School of Cyber Security and Computer. Her research interests include network security and searchable encryption technology.



Chunfu Jia is a Ph.D. Supervisor, a Professor, and the Head of Department with the Department of Cyber Sciences, Nankai University. His research interests include network and system security, cryptography application, and malware analysis.



Ruizhong Du received the B.S.E. and M.S.E. degrees in computer science from Hebei University, in 1997 and 2004, respectively, and the Ph.D. degree in information security from Wuhan University, in 2012. He is currently a Professor, a Ph.D. Supervisor, and the Dean with the School of Cyber Security and Computer, Hebei University. His research interests include network security, trusted computing, cloud security, and edge computing. He has published more than 60 research papers in refereed international conferences and journals, including IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE International Conference on Web Services, IEEE/IFIP International Conference on Dependable Systems and Networks, Computers & Security, and IEEE TRANSACTIONS ON CLOUD COMPUTING.



Guanxiong Ha received the M.S. degree in computer science and technology in 2021 from Nankai University, Tianjin, China, where he is currently working toward the Ph.D. degree with the College of Cyber Science. His research interests include encrypted deduplication, cloud data security, and applied cryptography. He has authored or co-authored several papers at premium international journals and conferences, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON CLOUD COMPUTING, IEEE ICWS, Information Sciences, and Asia CCS. He has been invited as the Reviewer for international journals and conferences such as IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING and IEEE ICC.